

Session 5

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 21-Sep-2026

Announcements

- Reminder — finish the Module 2 notebooks if you have not: m2_vectors_transforms_student.ipynb and m2_eigen_svd_pca_student.ipynb
- Refresher Module 3, Part 1 (PDF) is on Drive — read it before the lab if you can

Previously on CRB1

- Session 4: Refresher Module 2 (PDF) — vectors, projection, Gram-Schmidt, transformation zoo, eigen/SVD
- Notebooks 1–2 (Vectors & Transforms, Eigen/SVD/PCA)

Questions?

Today's Agenda

- Refresher Module 2 (PDF), §15: PageRank — ranking nodes by eigenvector centrality
- Refresher Module 3, Part 1 (PDF): the derivative and the chain rule, the three losses, the gradient and the tangent plane, the total differential, Taylor, backprop and why it runs backwards, gradient descent and the learning-rate law, autodiff
- Notebook 1 — PageRank: Ranking Nodes by Eigenvector Centrality
 - power iteration, damping, a brain-network capstone
- Notebook 2 — Derivatives, Gradients & Optimisation
 - steepest ascent and the tangent plane, the total differential, Taylor, the three losses, backprop and one layer's gradients, vanishing gradients, reverse vs forward mode, the learning-rate law, autograd
- Kahoot

Refresher Modules

Where today's session fits before Part I begins

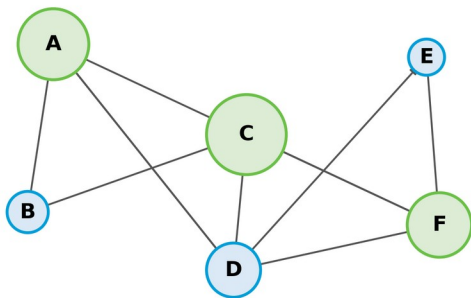
- Module 1: Python, NumPy, Pandas, Matplotlib — the data & compute layer (Sessions 2–3)
 - Module 2: Linear Algebra — vectors, transforms, eigen/SVD (Session 4); PageRank (today)
 - Module 3: Calculus — derivative, gradient, gradient descent, autodiff (today); integration, PID (Session 22)
 - Module 4: Image Processing (OpenCV) (Session 6)
 - Module 5: LLMs & AI Coding Assistants (Session 7)
-
- Training a model means minimising a loss — the derivative and the gradient are what make minimisation possible

PageRank

Refresher Module 2 (PDF), §15 — ranking a network by an eigenvector

- A page is important if important pages link to it, each splitting its importance across its outgoing links: $Lr = r$, the eigenvector equation with $\lambda = 1$
- L is column-stochastic, so its dominant eigenvalue is exactly 1 and power iteration $r \leftarrow Lr$ converges without ever computing an eigenvalue
- Dead ends and traps break it; damping $M = dL + (1-d)/n \cdot J$ repairs both. The identical algebra ranks genes in a regulatory network or hub regions in a brain-connectivity graph

Importance Is an Eigenvector: $Lr = r$



Bigger, greener node = higher PageRank; a page is important if important pages link to it

Let's practice — Notebook 1

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m2_pagerank_networks_student.ipynb

[Open Notebook 1 in Google Colab](#) →

The Derivative & Chain Rule

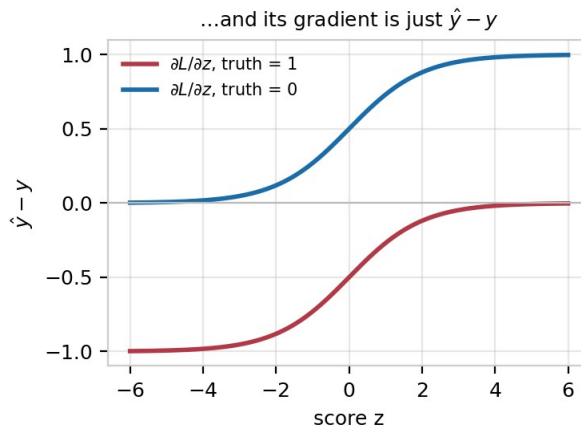
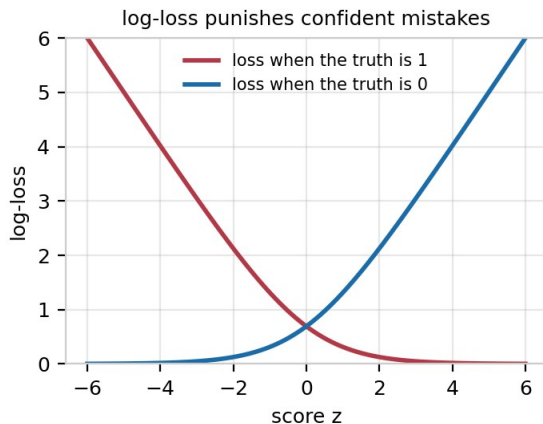
Refresher Module 3, Part 1 (PDF), §1 — the one rule that makes training possible

- The derivative $f'(x) = df/dx$ is a slope and a rate: in a model it says which way and how fast the loss moves when a parameter is nudged
- The chain rule is the rule that matters. A network is a deep composition $f_L(\dots f_2(f_1(x)))$, and training it means differentiating that composition layer by layer
- Squared error already shows the pattern: for $L = \frac{1}{2}(wx - y)^2$, $dL/dw = (wx - y) \cdot x$ — the error times the input, a shape that will keep coming back

The Three Losses

Refresher Module 3, Part 1 (PDF), §1.4 — and the one gradient shape they share

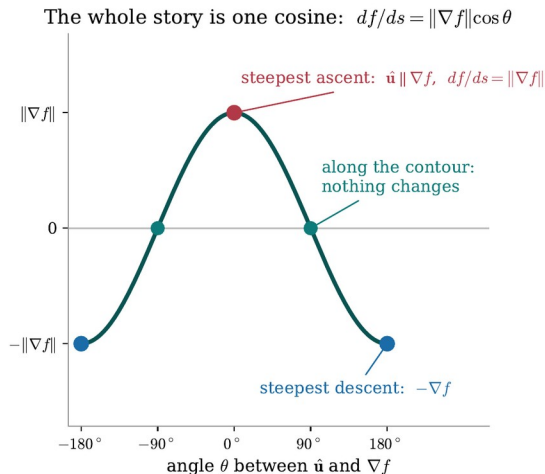
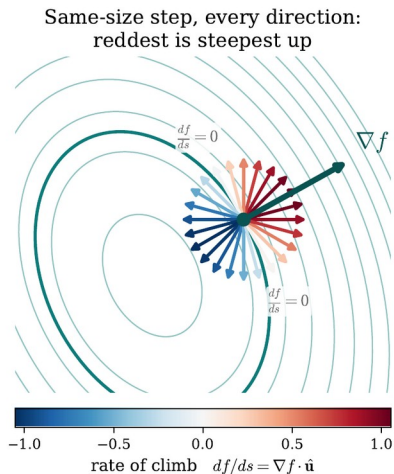
- Regression uses squared error with no activation; binary classification uses the sigmoid with log-loss; multi-class uses the softmax with cross-entropy
- All three give $\partial L / \partial z = \hat{y} - y$ at the scores. The activation's derivative and the loss's derivative cancel exactly — which is why these pairings are the standard ones
- That is also why frameworks fuse the softmax into the loss (CrossEntropyLoss takes raw scores): the fused gradient is a subtraction, so computing it separately is slower and numerically worse



The Gradient Points Uphill

Refresher Module 3, Part 1 (PDF), §3.3 — steepest ascent is one cosine

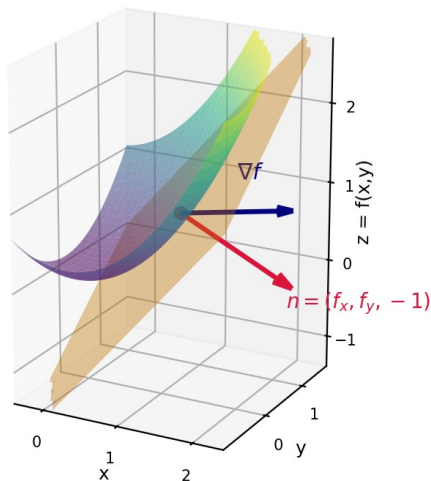
- Step a unit direction $\hat{u}$ and the rate of climb is $df/ds = \nabla f \cdot \hat{u} = \|\nabla f\| \cos \theta$. Every claim about the gradient is a value of that cosine
- $\theta = 0$ gives $\|\nabla f\|$, the largest available — so the gradient's direction is steepest ascent and its length is that rate. $\theta = 90^\circ$ gives zero: you are on a contour. $\theta = 180^\circ$ gives steepest descent, which is the whole justification for stepping along $-\nabla f$



The Tangent Plane

Refresher Module 3, Part 1 (PDF), §3.3 — where the -1 comes from

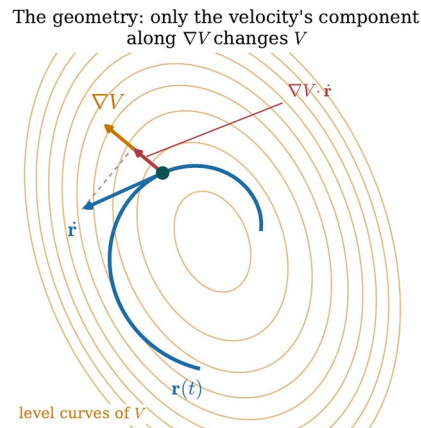
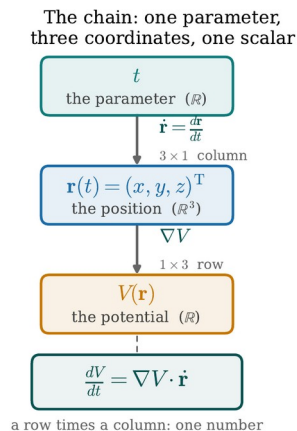
- $\nabla f = (f_x, f_y)$ lives in the domain, drawn on the contour map. The graph $z = f(x,y)$ is a surface in space, and near a point it has a tangent plane
- That plane's normal is $n = (f_x, f_y, -1)$ — the 2-D gradient with a -1 stapled on. Set $F(x,y,z) = f(x,y) - z$, so the surface is the level set $F = 0$; then $\nabla F = (f_x, f_y, -1)$, and a gradient is always perpendicular to its own level set
- The -1 is nothing exotic: it is $\partial F / \partial z$. Two pictures, one theorem



The Total Differential

Refresher Module 3, Part 1 (PDF), §3.2 — a path through a field

- A parameter picks a point, $\mathbf{r}(t)$; a scalar field assigns a number to every point, $V(\mathbf{r})$. Compose them and $dV/dt = \nabla V \cdot \dot{\mathbf{r}}$ — a 1×3 row times a 3×1 column
- Multiply through by dt and the parameter disappears: $dV = \nabla V \cdot d\mathbf{r}$, the total differential. Move along a contour ($d\mathbf{r} \perp \nabla V$) and nothing changes
- The shape scalar $\rightarrow$ vector $\rightarrow$ scalar is not a physics curiosity — it is backpropagation. Physics runs it forwards along a path; training runs it backwards from the loss

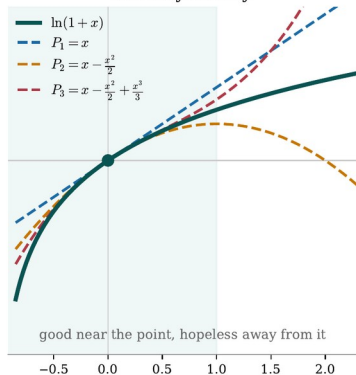


Taylor: the Local Model

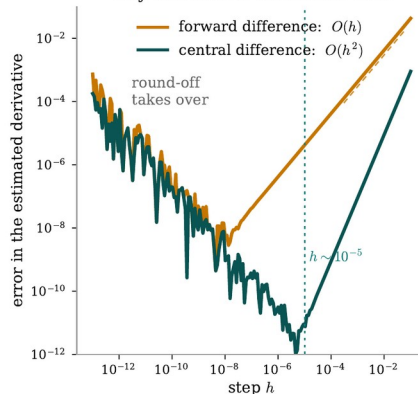
Refresher Module 3, Part 1 (PDF), §4 — where you truncate decides the algorithm

- Every optimiser works on a local polynomial model, not on the loss itself: $f(x+\delta) \approx f(x) + \nabla f^T \delta + \frac{1}{2} \delta^T H \delta$
- Truncate after the linear term and minimising the model is gradient descent — the step size η is the price of truncating early. Keep the quadratic term and it is Newton's method, which lands exactly on the minimum of a quadratic loss in one move
- The same expansion explains Section 2's error orders: the even terms cancel in the central difference, which is why it is $O(h^2)$ and the forward one only $O(h)$

Matching more derivatives buys accuracy, but only nearby



The same expansion, cashed out: why the central difference wins



The Jacobian & the Hessian

Refresher Module 3, Part 1 (PDF), §3.4–3.5 — first and second derivatives, stacked

- For a vector-valued function the Jacobian collects every first partial: $f(x+\delta) \approx f(x) + J\delta$. The gradient is just the Jacobian of a scalar function
- This is exactly the manipulator Jacobian of the Robotics & Bionics unit, where f is the forward kinematics and J relates joint velocities to end-effector velocity, $\dot{x} = J\dot{q}$
- The Hessian is the matrix of second partials — curvature. It is symmetric, so by the spectral theorem of Module 2 its eigenvalues are real: all positive is a bowl, mixed signs a saddle. Their ratio is what makes a loss hard to optimise

Backprop & One Layer

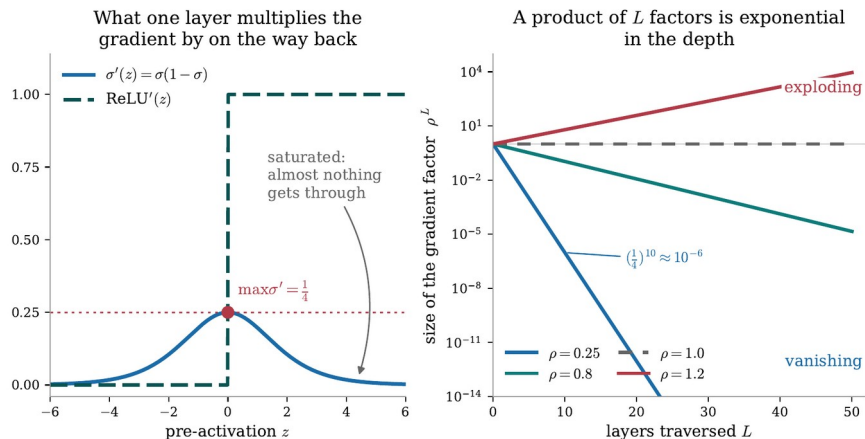
Refresher Module 3, Part 1 (PDF), §5–5.1 — the two formulas you will meet everywhere

- Backpropagation is the chain rule run backwards through a computational graph: each gradient is a product of local derivatives along the path from the loss back to the input
- For a real layer $z = Wx + b$, with $\delta = \partial L / \partial z$ already delivered: $\partial L / \partial W = \delta x^T$, an outer product, and $\partial L / \partial x = W^T \delta$, which is what gets passed further back. That is why transposes appear everywhere in backprop
- The shape rule saves hours: the gradient of a scalar with respect to anything has the same shape as that thing. If it does not match, the derivation is wrong

Vanishing & Exploding

Refresher Module 3, Part 1 (PDF), §5.2 — why depth is hard

- Backprop multiplies one factor per layer, so the gradient reaching the first layer is of size ρ^L — exponential in the depth. There is no comfortable middle
- The sigmoid is the classic offender: $\sigma' = \sigma(1-\sigma)$ never exceeds $\frac{1}{4}$, so ten stacked sigmoid layers shrink the gradient by 10^{-6} at best
- Read Géron's Chapter 11 with this in mind and its remedies stop looking like a list of tricks: ReLU, Glorot/He initialisation, batch norm, gradient clipping and residual connections are all attacks on the same exponent

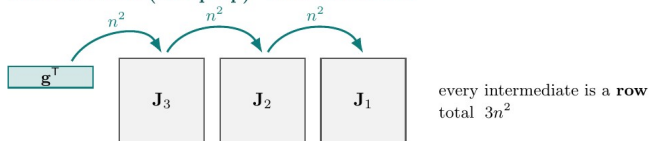


Backprop Runs Backwards

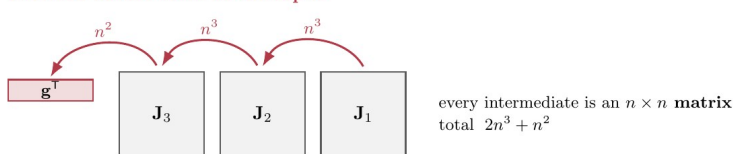
Refresher Module 3, Part 1 (PDF), §5.3 — multiply from the thin end

- For a stack of layers the chain rule is a product of Jacobians: $\partial L / \partial x = g^\top J_3 J_2 J_1$, with $g^\top = \partial L / \partial h_3$ a single row. Matrix products are associative, so it can be evaluated in either order
- From the loss end every intermediate is a row and each step costs n^2 (reverse mode = backprop). From the input end every intermediate is an $n \times n$ matrix and each step costs n^3 (forward mode). Same gradient, roughly n times the work
- The rule: start multiplying from the thin end. One scalar loss makes the output the thin end — hence `.backward()` on the loss. One input and many outputs flips it, and forward mode wins

Reverse mode (backprop): start at the loss



Forward mode: start at the input

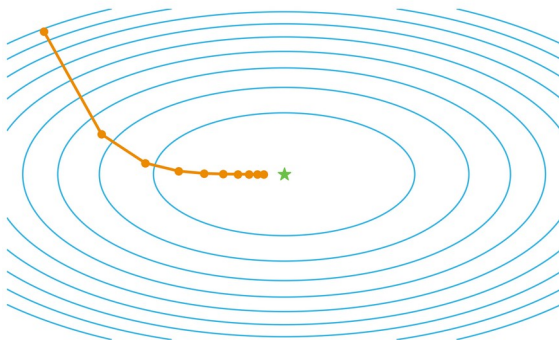


Gradient Descent

Refresher Module 3, Part 1 (PDF), §6 — stepping downhill

- $w \leftarrow w - \eta \nabla L(w)$: step against the gradient, with the learning rate η setting the size. Too small and training crawls; too large and it overshoots or diverges
- Why it works is the previous slide but one: truncate Taylor after the linear term and the model is decreased most by stepping along $-\nabla L$. Everything awkward about a fixed η lives in the quadratic term that truncation threw away
- A convex loss (linear or logistic regression) has one minimum that descent reaches. Deep networks are non-convex — many minima and saddles — yet descent still finds good solutions

Gradient Descent: $w \leftarrow w - \eta \nabla L(w)$

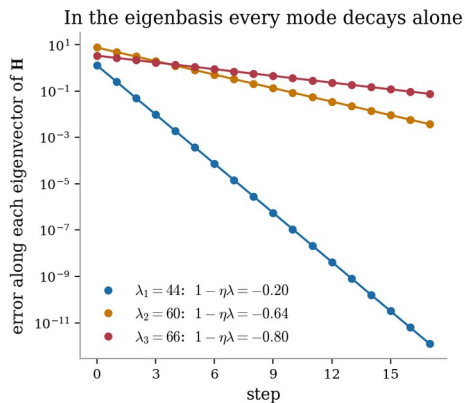
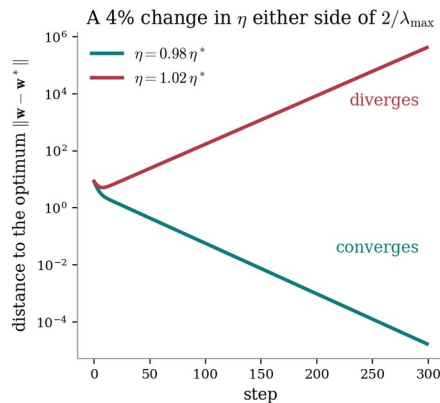


Each step moves opposite the gradient — downhill toward the minimum

The Learning-Rate Law

Refresher Module 3, Part 1 (PDF), §6.1 — read η^* off the Hessian

- Near a minimum the loss is a bowl, so one step acts on the error as $e \leftarrow (I - \eta H)e$. In H 's eigenbasis that matrix is diagonal — the easy-basis trick of Module 2 — and descent splits into independent modes, $c_i \leftarrow (1 - \eta \lambda_i) c_i$
- Every mode shrinks only if $|1 - \eta \lambda_i| < 1$, so descent converges exactly when $\eta < \eta^* = 2/\lambda_{\max}$. The threshold is sharp: 2% below converges, 2% above diverges
- Below it, the slowest mode shrinks by about $1 - 2/\kappa$ per step at best, $\kappa = \lambda_{\max}/\lambda_{\min}$. The law is local: for logistic regression the curvature falls as the model grows confident, so η^* read at the start is safe but conservative



Autodiff & PyTorch

Refresher Module 3, Part 1 (PDF), §7 — how a framework gets exact gradients

- Not by finite differences (too inaccurate) and not by symbolic algebra (the formulas explode), but by recording each elementary operation and applying the chain rule numerically, exactly, in one reverse pass — the backprop of §5, automated
- In PyTorch: mark a tensor `requires_grad=True`, call `.backward()` on a scalar loss, and every tensor's `.grad` holds the exact derivative. A training loop is then `loss, backward, step, zero the gradients, repeat`
- In the lab you compute one gradient three ways — by hand, by finite difference and by autograd — and watch them agree to machine precision

Let's practice — Notebook 2

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m3_derivatives_gradients_student.ipynb

[Open Notebook 2 in Google Colab](#) →

Before Next Session

- Finish both notebooks' exercises (solution cells included — check your own work)
- Re-read Refresher Module 3, Part 1 (PDF) if you haven't already — §1.4, §3.2, §3.3, §4, §5.1–5.3 and §6.1 are new
- Session 6 preview: Image Processing (Module 4 PDF)

Q & A



End of Session 5

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 21-Sep-2026